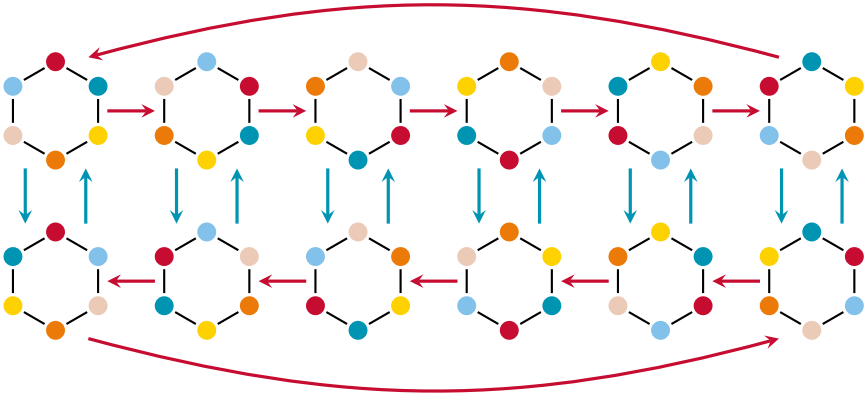
Module 7 project: Graph Isomorphism

Part IV: Automorphism Groups



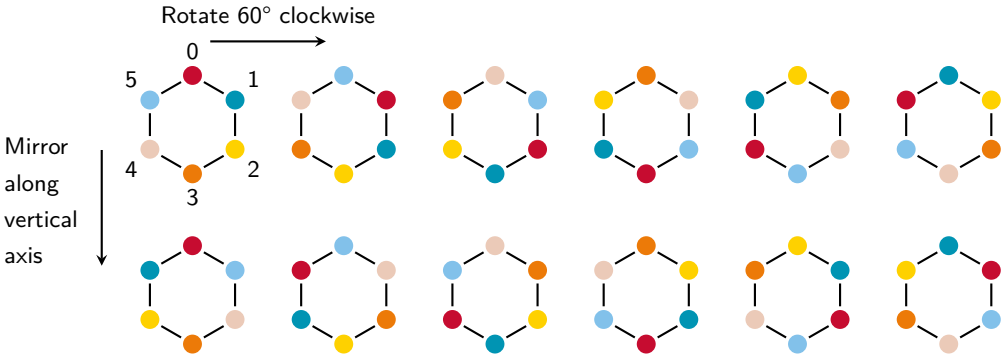
Ruben Hoeksma, slides partly by Paul Bonsma

New Ideas Are Necessary

- For deciding if two graphs G and H are isomorphic, can stop once one isomorphism has been found
- Now: count # of automorphisms of graph G by *generating them all*.
- That is not always feasible!
  (`bigtrees3.grl`, a tree on 227 vertices, with
  2 772 351 862 699 137 701 073 289 910 157 312 automorphisms ($\approx 2.7e33$)!)
  (For reference: age of universe, in milliseconds:
  $<$ 440 000 000 000 000 000 000 ($\approx 4.4e20$))
- Today: smarter methods for counting automorphisms of graphs.
- Necessary tools: Algebra.

Counting Automorphisms: Example

How many automorphisms does this graph have?

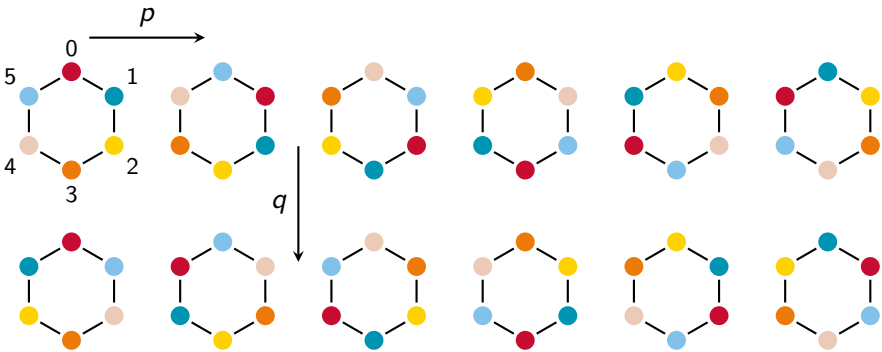


Find 12 automorphisms with two operations; these are *all* automorphisms.

Q: How to do this more formally / precisely?

A: Compute *generator* for $\mathrm{Aut}(G)$; working with *permutation groups*.

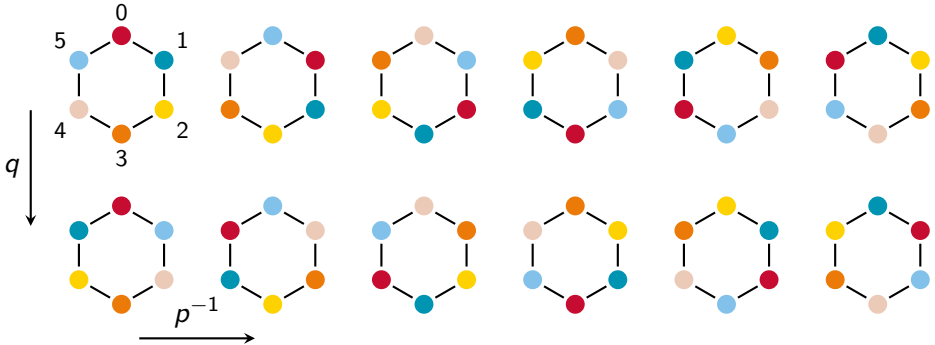
Automorphisms Using Permutations



Permutation	Operation	Mapping	Cycle Notation
p	Rotate 60° cw	$\begin{pmatrix} 0 & 1 & 2 & 3 & 4 & 5 \\ 1 & 2 & 3 & 4 & 5 & 0 \end{pmatrix}$	$(0\ 1\ 2\ 3\ 4\ 5)$
q	mirror vertically	$\begin{pmatrix} 0 & 1 & 2 & 3 & 4 & 5 \\ 0 & 5 & 4 & 3 & 2 & 1 \end{pmatrix}$	$(1\ 5)(2\ 4)$

$q \circ p = (0,5)(1,4)(2,3)$ $\qquad$ $p^{-1} \circ q = (0,5)(1,4)(2,3)$

Automorphisms Using Permutations



Permutation	Operation	Mapping	Cycle Notation
p	Rotate 60° cw	$\begin{pmatrix} 0 & 1 & 2 & 3 & 4 & 5 \\ 1 & 2 & 3 & 4 & 5 & 0 \end{pmatrix}$	$(0\ 1\ 2\ 3\ 4\ 5)$
q	mirror vertically	$\begin{pmatrix} 0 & 1 & 2 & 3 & 4 & 5 \\ 0 & 5 & 4 & 3 & 2 & 1 \end{pmatrix}$	$(1\ 5)(2\ 4)$

$q \circ p = (0,5)(1,4)(2,3)$ $\qquad$ $p^{-1} \circ q = (0,5)(1,4)(2,3)$

Automorphism Group Aut(G)

Observation (Inverse)

If p is an automorphism of G, then p^{-1} is an automorphism of G.

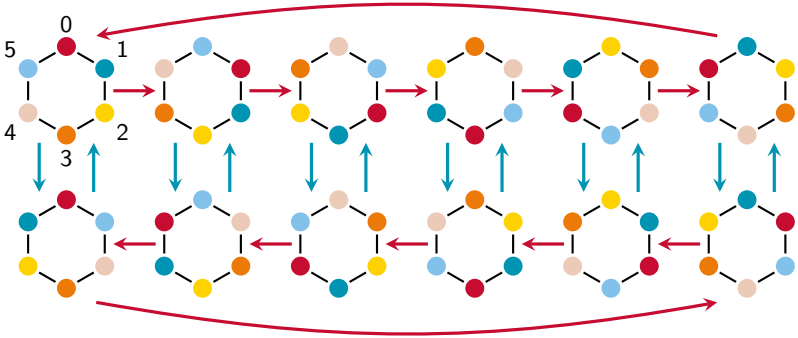
Observation (Composition)

If p and q are automorphisms of G, then $q \circ p$ is an automorphism of G.

- $\Rightarrow$ *Automorphisms of G form a group under function composition, Aut(G).*
  - *Closure:* $\forall p, q \in \mathrm{Aut}(G) : q \circ p \in \mathrm{Aut}(G)$.
  - *Identity:* $\forall p \in \mathrm{Aut}(G) : p \circ () = () \circ p = p$. *Trivial automorphism: ()*
  - *Inverse:* $\forall p \in \mathrm{Aut}(G)$: $\exists p^{-1}$ such that $p \circ p^{-1} = p^{-1} \circ p = ()$.
  - *Associativity:* $\forall p, q, r \in aut(G)$: $(r \circ q) \circ p = r \circ (q \circ p)$.
- Note: not *commutative:* often $p \circ q \neq q \circ p$!
- Let $V = V(G)$. Then Aut(G) is a permutation group, a subgroup of the *symmetric group on V, Sym(V)*, (= all permutations of V).
- Notation: $V(G) = \{0, \ldots, n-1\}$. (In line with Python)

Example (Again)

Using the automorphisms p and q, we can *generate* all automorphisms:



Definition

For set $S = \{p_1, \ldots, p_k\} \subseteq Sym(V)$. Then $H = \langle S \rangle$ is the subgroup generated by S; the smallest subgroup of $Sym(V)$ that contains all $p_i \in S$. S is a generating set of H.

Example: $\mathrm{Aut}(G) = \langle (0,3)(1,4)(2,5)\ ,\ (0,2,4)(1,3,5)\ ,\ (1,5)(2,4) \rangle$

Generating Set: Second Example

Let $G = K_n$, a complete graph on n vertices, with $V(G) = V = \{0, \ldots, n-1\}$.
Then $\mathrm{Aut}(G) = \mathrm{Sym}(V)$, $|\mathrm{Sym}(V)| = n!$.

Proposition

$Aut(G) = \langle (0, 1, \ldots, n-1)\ ,\ (0, 1) \rangle$.

Proof sketch: Using $p = (0, 1, \ldots, n-1)$ and $q = (0, 1)$, we can *invert* any two consecutive elements. (For inverting i and $i+1$, use: $p^i \circ q \circ p^{-i}$.)

By inverting consecutive elements, we can obtain any permutation. $\square$

Computing with Permutation Groups

Very large automorphism groups (order = exponential in n) can have very small generating sets (polynomial in n).
That is good news!

Questions:

- Is this true for any permutation/automorphism group?
- Can we find small generating sets efficiently?
- Can we use this for efficient computations?

Answers:

- Yes!
- In many cases: yes. (not always...)
- Yes!

$\rightarrow$ That is the goal for today.

Polynomial Size Generating Sets

The key to small generating sets: *only add permutations if needed.*

Observation

Let $H_1 = \langle p_1, \ldots, p_k \rangle$, $q \notin H_1$, *and* $H_2 = \langle p_1, \ldots, p_k, q \rangle$.
Then $|H_2| \geq 2|H_1|$.

Proof: H_1 is a subgroup of H_2. Consider the cosets of H_1 in H_2 (at least H_1 and qH_1), and use *Lagrange's theorem*. $\square$

So:

- Let V with $|V| = n$, and H be a subgroup of $\mathrm{Sym}(V)$.
- Any generating set $\{p_1, \ldots, p_k\}$ of H such that for all i, $p_i \notin \langle p_1, \ldots, p_{i-1} \rangle$, has *polynomial size*:

$$k \leq \log_2 |H| \leq \log_2 n! \leq \log_2 n^n \in O(n \log n).$$

Generating an Automorphism Group - Outline

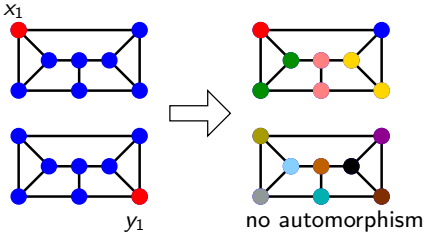
- How to find a small generating set for $\mathrm{Aut}(G)$ of a graph G ?
- Apply *individualization refinement* to $\{G, G'\}$, with G' a copy of G.
- Isomorphisms $f : G \to G'$ = automorphisms of G.
- Automorphisms are symmetries of G.
- Recall: # leaves in recursion tree $\geq$ # automorphisms of G.
- Goal: reduce this using the symmetries that we compute; maintain set X of automorphisms of G that (in the end) generates $\mathrm{Aut}(G)$.
- Use X to compute the number of automorphisms.
- Assumption: for $X = \{p_1, \ldots, p_k\}$, and permutation q, we can test whether $q \in \langle p_1, \ldots, p_k \rangle$: *Membership testing* (details: later.)

Individualization Refinement For Finding Automorphisms

- Let G be a graph, and let $D = [x_1, \ldots, x_\ell]$ and $I = [y_1, \ldots, y_\ell]$ be (equal length) sequences of vertices of G.
- We say that (D, I) is a *prescribed mapping*.
- We say that an automorphism $f : V(G) \to V(G)$ *follows* (D, I) if for all $i \in \{1, \ldots, \ell\}$, $f(x_i) = y_i$ holds.
- Notations: by $[\,]$ we denote the *empty sequence*, and by $D + x$ we denote appending element x at the end of sequence D.
- For given (D, I), colour refinement + branching may tell us:
  - (defines bijection) there exists at a unique automorphism that follows (D, I),
  - (unbalanced) there exists no automorphism that follows (D, I), or
  - (otherwise) undecided; then, choose a colour class of size ≥ 4, a vertex $x \in V(G)$ in that class, and for all y in *branching cell* $C :=$ vertices of $V(G')$ of that colour, recursively check the mapping $(D + x, I + y)$.
    Call x and C the *branching vertex* resp. *branching cell/colour*.

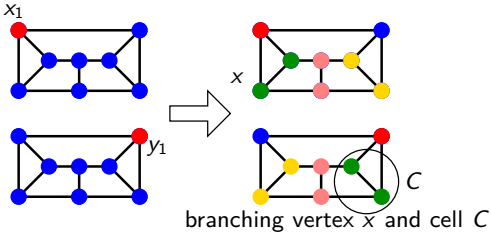
Example

Apply colour refinement to $\{G, G'\}$, with G' a copy of G.
The vertices in $D = [x_1, \ldots, x_\ell]$ and $I = [y_1, \ldots, y_\ell]$ are *individualized* in the start colouring $\alpha(D, I)$.
(For all i: x_i and y_i receive unique colour. Example: start with $D = [x_1]$, $I = [y_1] \ldots$)



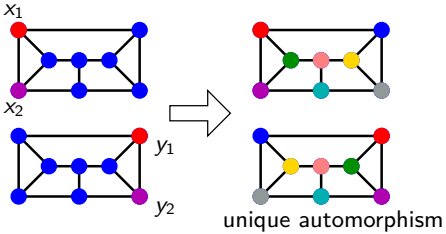
Example

Apply colour refinement to $\{G, G'\}$, with G' a copy of G.
The vertices in $D = [x_1, \ldots, x_\ell]$ and $I = [y_1, \ldots, y_\ell]$ are *individualized* in the start colouring $\alpha(D, I)$.
(For all i: x_i and y_i receive unique colour. Example: start with $D = [x_1]$, $I = [y_1] \ldots$)



Example

Apply colour refinement to $\{G, G'\}$, with G' a copy of G.
The vertices in $D = [x_1, \ldots, x_\ell]$ and $I = [y_1, \ldots, y_\ell]$ are *individualized* in the start colouring $\alpha(D, I)$.
(For all i: x_i and y_i receive unique colour. Example: start with $D = [x_1]$, $I = [y_1] \ldots$)



Basic Algorithmic Idea, I

INPUT: a graph $G = (V, E)$
OUTPUT: a (polynomial size) generating set X such that $\langle X \rangle$ is the automorphism group of G.

$X := \emptyset$
GENERATEAUTOMORPHISM([ ], [ ])
return X

Subroutine GENERATEAUTOMORPHISM(D, I):
INPUT: D and I are equal length sequences of vertices of G.
ACTION: Ensures that $\langle X \rangle$ contains all automorphisms f that follow D, I.

Basic Algorithmic Idea, II

First: a straightforward adaptation of the earlier branching algorithm.
We construct a generating set of $\mathrm{Aut}(G)$ by visiting all automorphisms.

Subroutine $\textsc{GenerateAutomorphism}(D, I)$:

Use colour refinement on $\{G, G'\}$ starting with colouring $\alpha(D, I)$.
- if there is no automorphism that follows (D, I) then return
- if there is a unique automorphism f that follows (D, I) then:
  if $f \notin \langle X \rangle$ then $X := X \cup \{f\}$ and return
- choose a branching vertex x and branching cell C with $|C| \geq 2$.

for all $y \in C$:
  $\textsc{GenerateAutomorphism}(D + x, I + y)$

Correctness

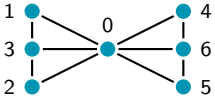
Lemma (Correctness)

The recursive call $\textsc{GenerateAutomorphism}(D, I)$ *ensures that* $\langle X \rangle$ *contains all automorphisms that follow* D, I.

Proof: By induction over the recursion tree. (Exercise) $\square$

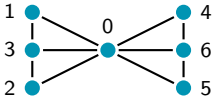
Key to efficiency: We can predict which branches do not lead to new automorphisms!

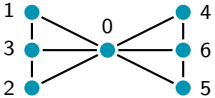
Example



Generating set for $\mathrm{Aut}(G)$: $\langle (4,5)\ ,\ (1,2)\ ,\ (3,6)(1,4)(2,5) \rangle$

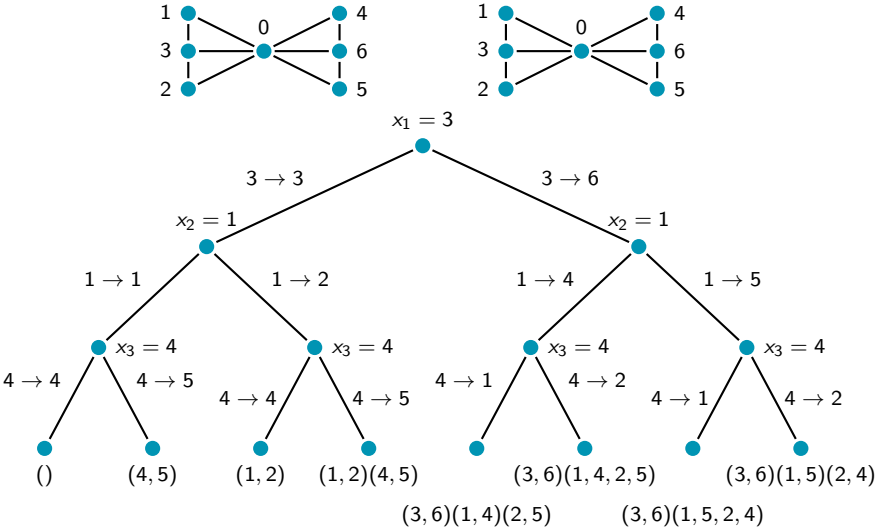
Example





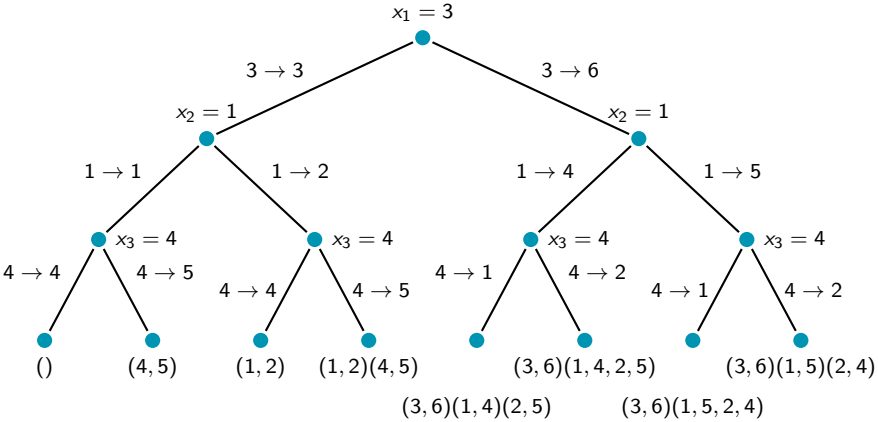
Generating set for $\mathrm{Aut}(G)$: $\langle (4,5)\ ,\ (1,2)\ ,\ (3,6)(1,4)(2,5) \rangle$

Example



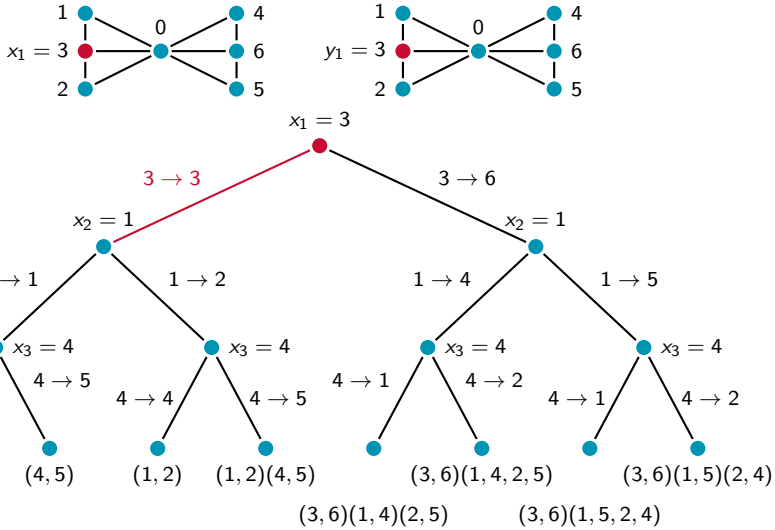
Generating set for $\mathrm{Aut}(G)$: $\langle (4,5)\ ,\ (1,2)\ ,\ (3,6)(1,4)(2,5) \rangle$

Example



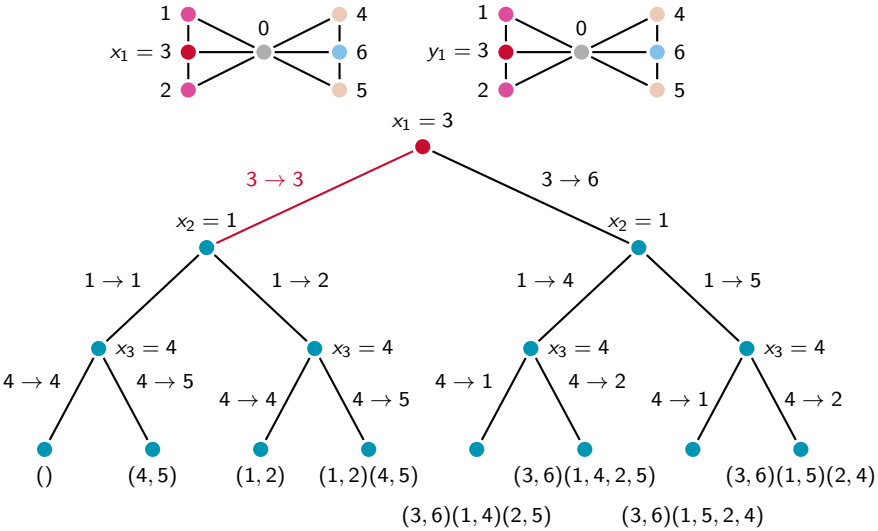
Generating set for $\mathrm{Aut}(G)$: $\langle (4,5)\ ,\ (1,2)\ ,\ (3,6)(1,4)(2,5) \rangle$

Example



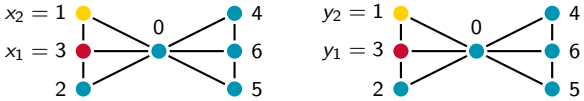
Generating set for $\mathrm{Aut}(G)$: $\langle (4,5)\ ,\ (1,2)\ ,\ (3,6)(1,4)(2,5) \rangle$

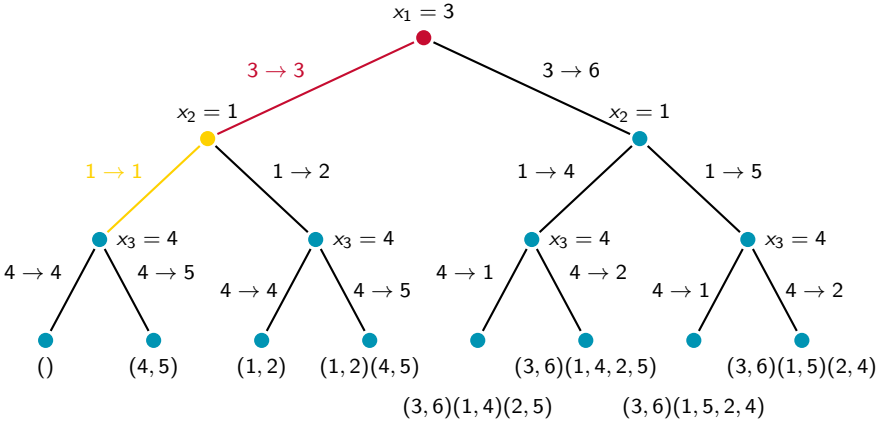
Example



Generating set for $\mathrm{Aut}(G)$: $\langle (4,5)\ ,\ (1,2)\ ,\ (3,6)(1,4)(2,5) \rangle$

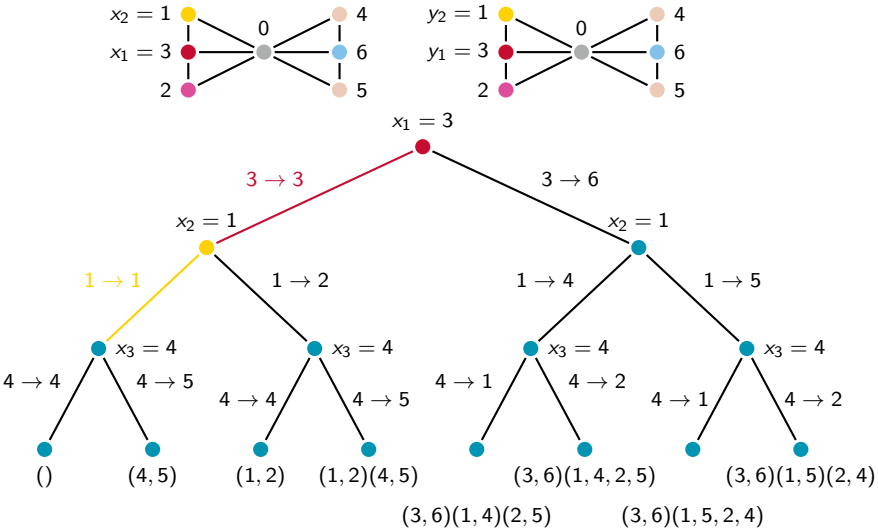
Example





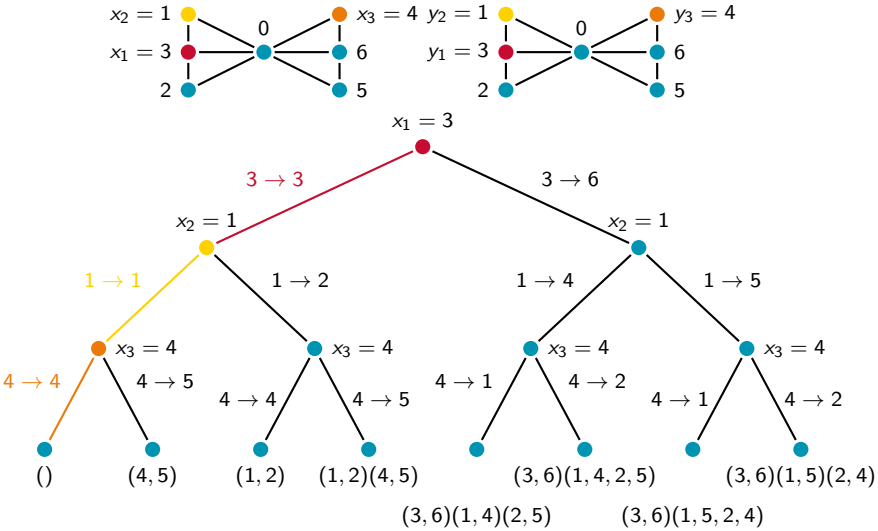
Generating set for $\mathrm{Aut}(G)$: $\langle (4, 5)\ ,\ (1, 2)\ ,\ (3, 6)(1, 4)(2, 5) \rangle$

Example



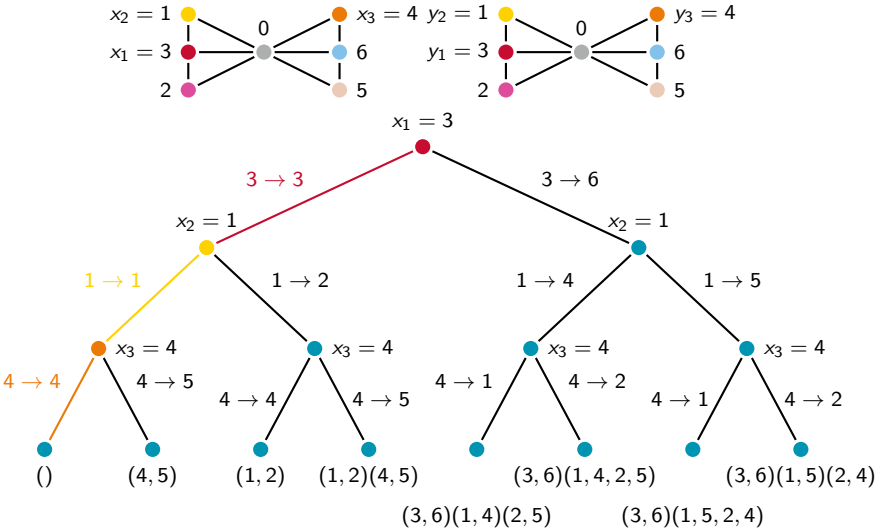
Generating set for $\mathrm{Aut}(G)$: $\langle (4, 5)\ ,\ (1, 2)\ ,\ (3, 6)(1, 4)(2, 5) \rangle$

Example



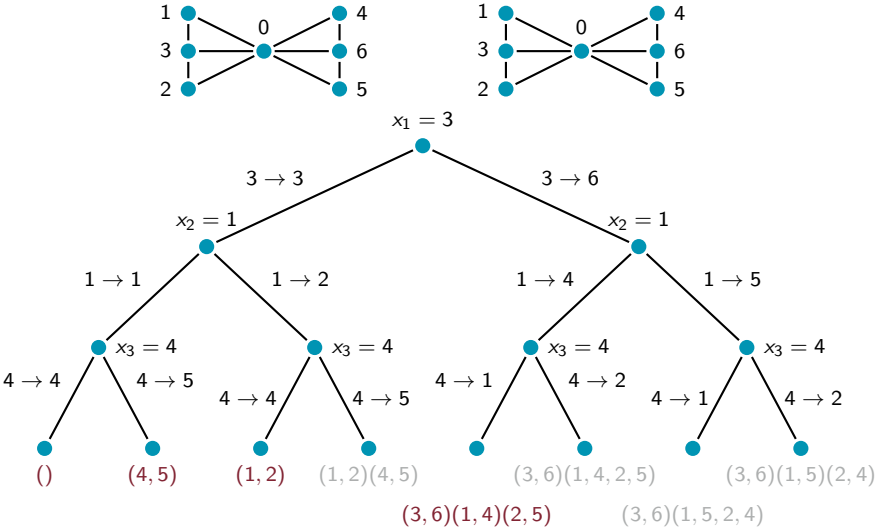
Generating set for $\mathrm{Aut}(G)$: $\langle (4, 5)\ ,\ (1, 2)\ ,\ (3, 6)(1, 4)(2, 5) \rangle$

Example



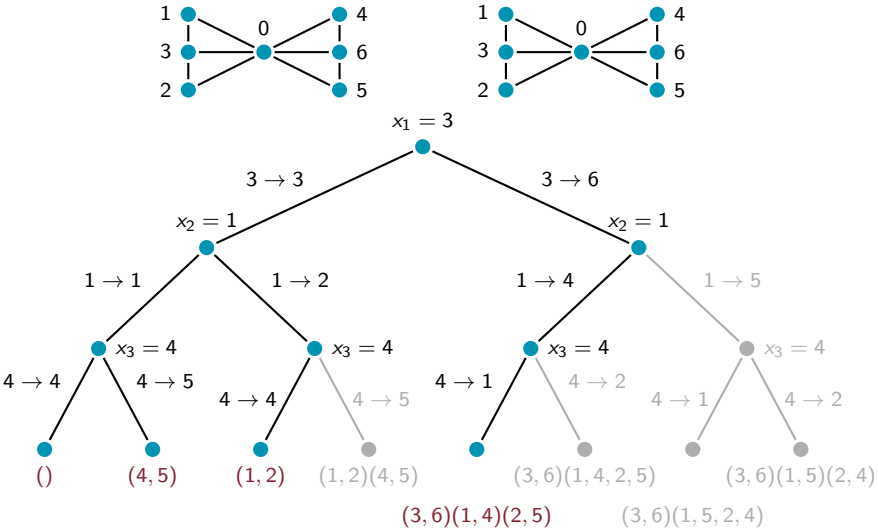
Generating set for $\mathrm{Aut}(G)$: $\langle (4,5)\ ,\ (1,2)\ ,\ (3,6)(1,4)(2,5) \rangle$

Example



Generating set for $\mathrm{Aut}(G)$: $\langle (4,5)\ ,\ (1,2)\ ,\ (3,6)(1,4)(2,5) \rangle$

Example



Generating set for $\mathrm{Aut}(G)$: $\langle (4,5)\ ,\ (1,2)\ ,\ (3,6)(1,4)(2,5) \rangle$

Pruning the Recursion Tree

Lemma (Pruning Lemma)

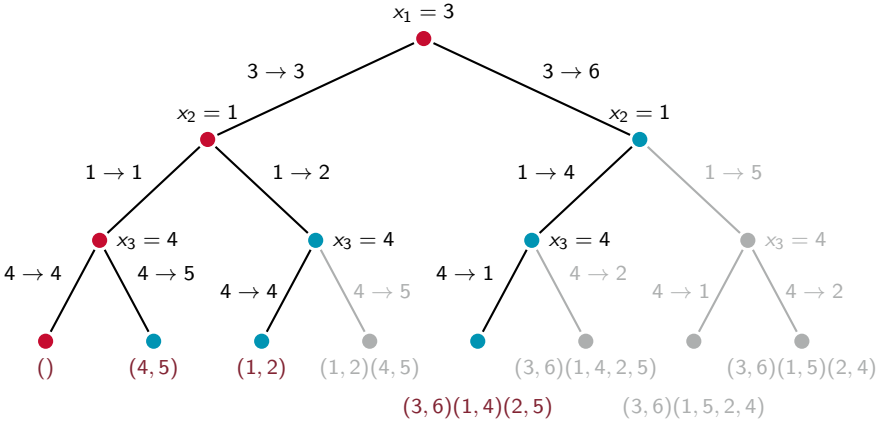
Let $\{x_1, \ldots, x_k, y\} \subseteq V(G)$. If subgroup H of $Aut(G)$ contains all automorphisms p with $p(x_i) = x_i$ for all $i = 1, \ldots, k$, and one automorphism q with $q(x_i) = x_i$ for all $i \leq k-1$ and $q(x_k) = y$, then H contains all automorphisms of G with the latter property.

Proof: Choose *any* automorphism $r \in \mathrm{Aut}(G)$ with $r(x_i) = x_i$ for all $i \leq k-1$ and $r(x_k) = y$.
Then $s := q^{-1} \circ r$ satisfies $s(x_i) = x_i$ for all $i = 1, \ldots, k$, so $s \in H$.
Therefore, $r = q \circ q^{-1} \circ r = q \circ s \in H$. □

Conclusion: Once we have all automorphisms that follow $(D + x, D + x)$ and only one automorphism that follows $(D + x, D + y)$, we may stop exploring this branch!

Call a prescribed mapping (D, D) *trivial*.
The corresponding node in the recursion tree is also called *trivial*.

Pruning the Recursion Tree



Pseudo Code: Pruning the Recursion Tree

Subroutine $\textsc{GenerateAutomorphism}(D, I)$:

Use colour refinement on $\{G, G'\}$ starting with $\alpha(D, I)$.

- if there is no automorphism that follows (D, I) then return
- if there is a unique automorphism f that follows (D, I) then:

  if $f \notin \langle X \rangle$ then $X := X \cup \{f\}$

  return to the last visited trivial ancestor node
- choose a branching vertex x and branching cell C with $|C| \geq 2$.

for all $y \in C$ (starting with $y = x$ if possible):

  $\textsc{GenerateAutomorphism}(D + x, I + y)$

(Further improvement possible by making predictions for more branches based on $\langle X \rangle$ but this is already very effective.)

Computational Problems for Permutation Groups

We know how we can find polynomial size generating set of $\mathrm{Aut}(G)$ *relatively efficiently*.

Two computational problems remain:

- Problem 1: *Order Computation:* given permutations $p_1, \ldots, p_k$, what is the *order* of the group $\langle p_1, \ldots, p_k \rangle$?
- Problem 2: *Membership Testing:* Given a permutation q and set of permutations $p_1, \ldots, p_k$, is $q \in \langle p_1, \ldots, p_k \rangle$?
- We will now see polynomial time algorithms for these questions.

Orbits, Stabilizers, and Transversals

Let H be a permutation group on V, and $\alpha \in V$.

- *Orbit* of α = all possible images: $\alpha^H = \{p(\alpha) \mid p \in H\}$.
- *Stabilizer of α in H* is the subgroup $H_\alpha = \{p \in H \mid p(\alpha) = \alpha\}$.
- A *transversal of H modulo H_α* is $\bigcup_{\beta \in \alpha^H} u_\beta$ with $u_\beta \in H$ and $u_\beta(\alpha) = \beta$ (For all $\beta \in \alpha^H$, one u_β that maps α to β.)

Example: $V = \{0, 1, \ldots, 5\}$
$H = \langle p, q \rangle$ with $p = (0,1,2)(4,5)$ and $q = (2,3)$. Let $\alpha = 0$.
The orbit of 0 is $0^H = \{0,1,2,3\}$, which is certified by (for instance) the following *tranversal elements:*

$$u_0 = (), \qquad u_1 = p, \qquad u_2 = p^{-1} = (0,2,1)(4,5), \text{ and}$$
$$u_3 = q \circ p^{-1} = (0,3,2,1)(4,5).$$

The stabilizer of 0 in H can be given as e.g. $H_0 = \langle (4,5)\ ,\ (2,3)\ ,\ (1,3) \rangle$.
$((4,5) = p^3$ and $(1,3) = p^{-1} \circ q \circ p.)$

Recursive Order Computation

For *computing the order of H*, we can iteratively use the following proposition (Proof with Lagrange's Theorem):

Proposition (Orbit-stabilizer Theorem)

For any $\alpha \in V$, $|H| = |H_\alpha| \cdot |\alpha^H|$.

Example, continued:
$H = \langle p, q \rangle$ with $p = (0,1,2)(4,5)$ and $q = (2,3)$.
Choose α with *nontrivial orbit:* $\alpha = 0$.

- *Orbit of α:* $0^H = \{0,1,2,3\}$, $|0^H| = 4$.
- *Stabilizer of α in H:* $H_0 = \langle (4,5)\ ,\ (2,3)\ ,\ (1,3) \rangle$.
- *Recursively,* we compute that $|H_0| = 12$. Base case: no non-trivial orbits.
- Therefore, the orbit-stabilizer Theorem gives: $|H| = |H_0| \cdot |0^H| = 12 \cdot 4 = 48$.

Recursive Membership Testing

For *testing membership* (is new $f \in H$?), we use the following:

Proposition (Sifting)

For $\beta \in \alpha^H$, let $u_\beta \in H$ with $u_\beta(\alpha) = \beta$ (transversal element).
For any permutation f with $f(\alpha) = \beta$, $f \in H$ if and only if $u_\beta^{-1} \circ f \in H_\alpha$.

Proof: Obviously, $u_\beta^{-1} \circ f$ maps α to α, so if $f \in H$ then $u_\beta^{-1} \circ f \in H_\alpha$.

Other direction: if $u_\beta^{-1} \circ f \in H_\alpha$ then $f = u_\beta \circ u_\beta^{-1} \circ f \in H$. $\square$

Recursive Membership Testing

For *testing membership* (is new $f \in H$?), we use the following:

Proposition (Sifting)

For $\beta \in \alpha^H$, let $u_\beta \in H$ with $u_\beta(\alpha) = \beta$ (transversal element).
For any permutation f with $f(\alpha) = \beta$, $f \in H$ if and only if $u_\beta^{-1} \circ f \in H_\alpha$.

Example, continued:
$H = \langle p, q \rangle$ with $p = (0,1,2)(4,5)$ and $q = (2,3)$. Choose $\alpha = 0$.

- *Orbit of α:* $0^H = \{0,1,2,3\}$.
  Corresponding tranversal elements:
  $$u_0 = (), \qquad u_1 = p, \qquad u_2 = p^{-1} = (0,2,1)(4,5), \text{ and}$$
  $$u_3 = q \circ p^{-1} = (0,3,2,1)(4,5).$$
- Testing whether $f = (0,2) \in H$:
- By Proposition: $f \in H$ if and only if
  $u_2^{-1} \circ f = (0,1,2)(4,5) \circ (0,2) = (1,2)(4,5) \in H_0$.
- *Recursively*, we compute that $(1,2)(4,5) \in H_0$

Summary / Remark on Computation Time

Let $X := \{p_1, \ldots, p_k\}$ be automorphisms found so far, $H := \langle X \rangle$, and new permutation f is found. To see if $f \in H$, choose element α with $|\alpha^H| \geq 2$, then

- either $f(\alpha) = \beta$ with $\beta \notin \alpha^H$, then $f \notin H$
- or, use (recursive) sifting: $f \in H \Leftrightarrow u_\beta^{-1} \circ f \in H_\alpha$

If we always choose α with non-trivial orbit (> 2 elements), the recursive computation terminates in at most $\log_2 |H| \leq \log_2 n! \leq n \log_2 n$ iterations, where $n = |V|$.

Most of these computations are also necessary for order computation. If you do them, don't do them twice!

Remaining questions:

- How do we compute orbit α^H and transversal elements u_β? (Easy)
- How do we compute generating set for H_α? (Not so easy)

Computing Orbit and Transversal

INPUT: generating set $\langle p_1, \ldots, p_k \rangle$ of H, element $\alpha \in V$.
OUTPUT: orbit α^H, and transversal elements u_β for all $\beta \in \alpha^H$.

$u_\alpha :=$ () (The trivial permutation)
$\alpha^H := \{\alpha\}$
$U := [\alpha]$ (a queue)
while U nonempty:
    $\beta :=$dequeue(U)
    for $i \in \{1, \ldots, k\}$:
        $\gamma = p_i(\beta)$
        if $\gamma \notin \alpha^H$:
            $\alpha^H := \alpha^H \cup \{\gamma\}$
            enqueue γ to U
            $u_\gamma := p_i \circ u_\beta$

One can (easily) show that this is correct, and a polynomial time algorithm.

Computing Orbit and Transversal: Example

$H = \langle p, q \rangle$ with $p = (0, 1, 2)(4, 5)$ and $q = (2, 3)$. Let $\alpha = 0$.

$0^H = \{0\}$
$U = [0]$

Element:	Transversal:
0	()

Consider $0 \in U$: $p(0) = 1$, so add 1 to orbit.
($q(0) = 0$: already present)

Computing Orbit and Transversal: Example

$H = \langle p, q \rangle$ with $p = (0, 1, 2)(4, 5)$ and $q = (2, 3)$. Let $\alpha = 0$.

$0^H = \{0, 1\}$
$U = [1]$

Element:	Transversal:	
0	()	
1	p	$= (0, 1, 2)(4, 5)$

Consider $1 \in U$: $p(1) = 2$, so add 2 to orbit.
($q(1) = 1$: already present)

Computing Orbit and Transversal: Example

$H = \langle p, q \rangle$ with $p = (0, 1, 2)(4, 5)$ and $q = (2, 3)$. Let $\alpha = 0$.

$0^H = \{0, 1, 2\}$
$U = [2]$

Element:	Transversal:	
0	$()$	
1	p	$= (0, 1, 2)(4, 5)$
2	$p \circ p$	$= (0, 2, 1)$

Consider $2 \in U$: $q(2) = 3$, so add 3 to orbit.
($p(2) = 0$: already present).

Computing Orbit and Transversal: Example

$H = \langle p, q \rangle$ with $p = (0, 1, 2)(4, 5)$ and $q = (2, 3)$. Let $\alpha = 0$.

$0^H = \{0, 1, 2, 3\}$
$U = [3]$

Element:	Transversal:	
0	$()$	
1	p	$= (0, 1, 2)(4, 5)$
2	$p \circ p$	$= (0, 2, 1)$
3	$q \circ p \circ p$	$= (0, 3, 2, 1)$

Consider $3 \in U$:
$p(3) = 3$ and $q(3) = 2$ are already present, so we are done.

Generating a Stabilizer Subgroup

The final ingredient:

Theorem

Let $\{p_1, \ldots, p_k\}$ be a set of permutations on V, with $|V| = n$, let $H = \langle p_1, \ldots, p_k \rangle$, and $\alpha \in V$.
In time polynomial in n and k we can compute a generating set for the stabilizer subgroup H_α, that contains at most n^2 permutations.

Proof: Omitted. Search for Schreier's Lemma if you're interested.

Good news: On Canvas, a Python program is given that does all of this for you!

Summary

- Given a generating set of a permutation group, have efficient algorithms for *Order Computation* and *Membership Testing*.
- For finding a generating set of $\mathrm{Aut}(G)$, can use *individualization refinement*, but a lot of the branches of the recursion tree can be pruned, which makes this *really fast* in practice!

Project: Practical Remarks

- `permv2.py` : A module for working with permutations.
  See the documentation for more information on usage.
- `basicpermutationgroup.py`: Some permutation group algorithms:
  - finding orbit and transversal, and
  - finding a generating set for a stabilizer.

  See the documentation for more information.

What you have to do yourself (if you want to implement this):

- Implement algorithms for:
  - Order Computation and
  - Membership Testing.
- Use your branching algorithm for finding automorphisms, and add pruning rules.
  (Test whether the outcome is still the same!)
- Membership testing is a trade-off between fast branching and fast order computation. Test if your algorithm is faster with/without.